
WHITE PAPER · AI OPERATING GOVERNANCE

An AI Evaluation Is a Production Operating Control

Govern changing workflow behavior, not a one-time test

Marco Policani

Enterprise Portfolio · PMO · AI Operating Governance

policani.net · linkedin.com/in/marcpolicani

July 2026

EXECUTIVE SUMMARY

Treat “the system works” as true only as of the last evaluation, and retest whenever the model, data, or workflow changes.

A model can pass a launch evaluation while its workflow keeps changing. New cases, data, and model versions can make that result old. A production evaluation checks the live cases and gives a named owner evidence to limit, pause, or restore use.

- 01** A benchmark compares models, a red team seeks weak points, and a production evaluation tests the cases this team handles today.
- 02** Set a passing limit for each type of case before testing, stating what result counts as a pass, with stricter limits where errors cost more or overwhelm human reviewers.
- 03** Compare the last evaluation date with the last material change; a newer change calls for retesting or limiting use until the evidence catches up.

The argument

Consider a workflow that passes a careful launch test with real cases and clear limits. Later, the vendor updates the model, another team changes the input format, and the business adds a case type the test never covered. Each change may seem reasonable alone. If no one checks their combined effect, poor results may go unnoticed until a complaint reaches the team. The launch test was sound when it ran. What now lets this team rely on the changed workflow?

The old result did not cover the new case type or the changed system. A passing result supports use only for the cases and conditions the test checked. A benchmark can help choose a model, and a red team can search for weak points. Neither answers whether today's workflow still works for its users. That requires a current test of real work and an owner who can limit use when the evidence no longer fits.

A model can pass a test and still fail the team using it because the test captured one point in time. New cases appear, users depend on outputs in new ways, source data ages, and vendors change the model. A passing score from last quarter may no longer describe the system the team uses today. Research on machine learning

shows how changing data and system ties create hidden upkeep costs.[3] Research on generated text also shows that a fluent answer can be wrong,[4] so neither an old score nor polished wording proves that the current workflow is sound.

This paper brings those checks into a production evaluation loop with seven parts. NIST's generative-AI profile calls for continuing measurement, and Microsoft's red-team lessons show why finding failures takes skilled, sustained effort.[1][2] The loop connects testing to decisions about use. It is for AI product leaders, workflow owners, risk teams, and technology executives.

Three tests, three questions

The word "evaluation" covers three jobs that a team should keep separate when it makes decisions about use. A benchmark compares models on common tasks to help the team choose an engine, while a red team searches for ways to make one fail. A production test checks the actual workflow, including its cases and the cost of errors. It asks whether this team can rely on this system for these cases today.

Each test has a limit: a strong benchmark cannot prove a workflow is safe, and a red-team exercise at launch cannot show that it still works a year later. A production test cannot approve new case types it never covered. Teams stretch results because testing takes time and money, but that shortcut lets people rely on a system beyond the evidence. Microsoft Research distinguishes red teaming from safety benchmarking and treats the work of finding weaknesses as continuing.[2]

Keep a short record for each test: the question it answers, the cases it covers, the setting in which it ran, and the decision it can support. If a decision needs another answer, run the missing test. Do not claim that an old result covers more than it did.

The three tests need different budgets. Public benchmarks can help teams shortlist models. Red teams need skilled people and should return after major changes or when exposure calls for another search.[2] The workflow team must keep its own production test current. Only that team knows its cases, the cost of errors, and what acceptable work looks like. A single budget line for all three can leave the daily test unfunded.

The production evaluation loop

The loop has seven disciplines. The first three build the evaluation's substance: cases, thresholds, and the separation above. The next three keep it alive: monitoring, change events, and incident learning. The last connects it to power: someone must

be able to change what the workflow relies on, or the apparatus is measurement without consequence.

EXHIBIT 1

Keep the test current

1. Cases from real work

Work history shows which case types the evaluation covers.

2. Limits by case type

Set passing limits before results, stricter where errors cost more.

3. Keep the tests apart

A benchmark, a red team, and a production test answer different questions.

4. Watch live signals

Changes in inputs, answers, and reviewer corrections tell the team when to look.

5. Retest after change

Rerun the cases a model, data, or workflow change could affect.

6. Learn from failures

Turn failures and near misses into new cases, then check the fix holds.

7. Owner who can act

The workflow owner reads results and warnings, then limits, pauses, or restores use.

Build true-to-life workflow cases

The test cases decide what the result can mean. Convenient demos show the system at its best. Real work includes messy inputs, rare but costly cases, and errors people are likely to make. Draw cases from work history. Give extra attention to groups where an error costs more. Add likely misuse found by red teams. Then the test can speak to the work people actually do.

Record where each group of cases came from. Does its share of the test match the real workload? Which reference answers are clear, and which need human judgment? People who do the work should help choose and label cases. They know the hard ones. Their judgment is more useful than a large set of invented examples.

The case set also sets a limit on use. The team can rely on the system only for the kinds of cases it tested. If the business adds a new case type, the team must add cases, check the results, and then decide whether to allow that use. The order matters.

A larger case set is not always a better one. A smaller set drawn from real work, with clear reference answers, may reveal more useful failures than a large set of scraped examples. The team still has to read those failures and act on them. Start with cases the workflow actually sees, then add cases where incidents show a gap.

Define thresholds by consequence

An average can hide a dangerous class of errors. For example, a system could score 96 percent overall while failing three of every ten critical cases. Set a limit for each case type based on the cost of failure. Routine cases may allow more errors, while critical cases need a stricter limit and some answers must go to a person. Include the load on human reviewers and delays when timing matters, because both affect whether the workflow can use the system safely.

The system's choice to send a case to a person, rather than answer it, needs its own limit. If the system answers everything, it may make confident mistakes. If it sends too much back, it shifts the work onto reviewers. Set an expected range for each case type and watch both ends. That catches overconfidence and a loss of useful performance.

Write the thresholds before seeing results. Afterward, they negotiate themselves toward whatever the system achieved, and the evaluation becomes a mirror instead of a bar. Writing them first forces the uncomfortable, valuable conversation about what the operation actually requires: which failures are absorbable, which are expensive, and which are the reason the workflow has a human in it at all.

The decision rule keeps the aggregate honest: a critical-class failure pauses or narrows reliance even when the average looks excellent. Review load belongs in the threshold set too. False rejections that flood human reviewers are a real cost, and a threshold structure that ignores them will be quietly overridden by the operation defending its own capacity.

Monitor production signals

Between formal evaluations, the workflow leaves warning signs if the team looks for them. Incoming cases change. Answers shift. Reviewers correct more results. Errors

rise in one case type. Users find workarounds. A team that tracks only uptime and cost can see the system running, but cannot tell whether it still fits the work.

The team should check whether the system still fits the work. Compare incoming cases with the test set, look for changes in answers, record reviewer corrections, and sample live results. A regular sample read by someone who knows the work may be a useful start. Set its size and frequency to match the risk, then add checks where warnings show a gap. Keep the person who can act on those warnings in the loop.

Decide who acts on each warning by giving every signal a threshold and a named owner. When it trips, that owner can test the affected cases, check the review team's workload, or add new cases. If the warning puts current use in doubt, the owner brings it to the person who can limit or pause that use. A dashboard without these actions only explains the failure after it happens.

Reviewer disagreement is another warning. If two skilled reviewers judge the same live cases differently, either the system or the standard may have shifted. Compare their calls in the monthly sample, then examine the disputed cases. Human review has limits too. Research finds that people can miss failures while watching reliable automation alongside other work, and that teams can share the same bias.[5] A forced second look may catch an error that another passive reviewer would miss.

Turn change into a test event

A model update, prompt edit, new data source, tool, policy, or workflow change can make old test results less useful. The team should ask which cases each change could affect and retest them before people rely on the new version. A new group of users may require new cases too.

Retest according to what changed. A vendor model update may affect many case types; test all the ones it could change, or run the full set if the vendor cannot explain the update. A prompt edit may affect fewer cases. A new data source needs tests that use it and a check for bad or misleading inputs. A redesigned workflow may need new cases. Keep a version record so the team can see which test covers the system now in use.

If a changed case has not been retested, wait before using the new version for that work. If the new test falls below its limit, narrow use until the problem is fixed. Teams often accept this rule for code but overlook prompts and models, even though those changes can alter answers too.

Ask vendors about change before the team depends on them. Can they warn you before a model update, let you keep a version for a time, and explain what changed?

The answers vary. If a vendor cannot explain an update, the team may need to rerun every case. That testing cost belongs beside the vendor's usage price.

Learn from incidents and near misses

A failure or near miss gives the team a new test case, but first it needs to find what caused the error. Check the model, data, screen, instructions, review load, access, and pressure on staff. Add a case that would catch the same failure next time, change a warning limit if it missed the signal, and track whether the error returns. A one-off prompt fix can leave the wider cause in place.

That review's breadth is what makes it worth running, because the cause is frequently not the model at all. A misrouted case may trace to an upstream data change. An accepted bad answer may trace to a review queue running at twice its designed volume. Converting incidents into cases without that breadth just grows the case set while the actual defect recurs untouched.

Record near misses as well as failures. A reviewer who catches a bad answer has shown that the review worked and that the system made an error. Both facts belong in the record; the next reviewer may not catch the same mistake. If the team waits for harm before learning, it loses a chance to correct the workflow earlier.

Over time, cases drawn from actual failures make the evaluation stronger. The case set stops reflecting only what the launch team expected and starts recording what the operation has faced. Another workflow can use those lessons when the same failure type appears, but it still needs to test its own cases.

Assign authority to change reliance

Tests and warnings matter only if someone can change what the team is allowed to do with the system. The workflow owner decides which uses are allowed, while a technical owner tracks the system and its versions. A risk reviewer can challenge the decision. The group reviews results on a set schedule and has a faster route when an urgent finding cannot wait.

A common failure is a report everyone reads but no one uses. A test misses its limit, the finding sits in an appendix, and the team keeps using the system as before. When no one can connect a warning to a change in allowed use, the company has test results but no working control.

The record should show what changed after a warning, who made the decision, and which test result supported it. Did the owner limit use within days of a failed threshold

and restore it only after another test? Has the team practiced stopping the workflow and using its backup process? The key measure is how long the team takes to act on a signal.

A regular review closes the loop by bringing test results, warnings, system changes, and incidents to the people who can act. Each month, or at a pace that fits the risk, the group records one result for each workflow: keep current use, change it, or name an evidence gap with an owner and date. A meeting with no decision is only a briefing.

The loop as a system

The parts support one another. Cases from real work give passing limits meaning. Those limits help the team act on warnings. Change checks keep the cases current. Failures add new cases, and a named owner decides what to do with the results. If the team builds cases and limits but does not watch for change, it may discover only after a failure that its old test no longer describes the workflow.

Other AI controls depend on current test evidence. A team should give the system more responsibility only for cases the tests support. Reviewers may catch one bad answer, while ongoing checks can reveal a wider pattern. The decision made at launch stays sound only if the team checks again as the system and work change.

Standing the loop up

The loop installs in the order its evidence compounds:

- Create the first case set from real workflow history, not a generic benchmark.
- Write acceptance and stop thresholds before running the evaluation.
- Track live warning signs and reviewer corrections in one list of cases to check.
- Retest after a material change, and review results on a set schedule even when nothing fails.

The first case set can teach the team about its own work. Work history reveals exception types nobody counted, cases that use much of the reviewers' time, and quiet workarounds people rely on. The team may learn these things before it runs the first test. That lesson helps it choose cases worth keeping.

The loop takes regular work: keeping cases current, reading warnings, rerunning tests, and learning from incidents. Fund that work in line with how much the team depends on the system. If the team cannot fund the checks, it should limit how much

it relies on the system. Without an owner and time, testing stops while the last passing score keeps circulating.

The product team, platform group, or quality team can maintain the tests. The workflow owner still decides how much the team may rely on the system, and a risk reviewer should be able to challenge that decision. Builders can help test their system, but they should not be the only people who judge whether it passed.

What the record should show

A living evaluation leaves a current trail:

- A case set that records where cases came from, which groups it covers, and which uses it does not.
- Passing limits for each case type, set before results and checked against human review workload.
- Live warnings, the limit for each warning, and what the owner did when it fired.
- Changes to the model, data, or workflow matched to retests of the affected cases.
- Failures and near misses added as new cases, with a check that the same error did not return.
- Dates when the owner limited, paused, or restored use, and the test results behind each choice.

Buyers and auditors may ask for this trail, because dated test records show what the current system can do and what changed since the last check. Without the records, people must reconstruct the answer in meetings. That benefit is secondary to the main purpose: protecting the work that now depends on the system.

A quick date check can expose stale evidence. Compare the last evaluation with the last material change to the system, data, or workflow. If the change came later, record which cases still have support and which need another test. Give the gap an owner and a date. The loop makes that gap visible so the team can limit use or close it.

The strongest counterargument

No evaluation can prove universal safety or future performance. It provides bounded evidence for a named use under observed conditions. Honest limits, maintained cases, and authority to change reliance are worth more than a large one-time score.

EXHIBIT 2

The cost of keeping evidence current**Keep checking**

Set regular review time based on how much the team relies on the system and what an error would cost.

Stop checking

A missed decline can last weeks until a failure exposes it and forces a rushed review.

Keep a dated record

Dated test records show buyers and auditors what the current system was checked against.

Check for a gap

Compare the last test date with the last system, data, or workflow change, then investigate any gap.

A small team may see ongoing tests as too much work, yet skipping them can cost more when quality slips for months or a failure forces a rushed review. The team should size the checks to the risk and to how much it depends on the system. Even a small team needs a way to notice material change and act on it.

From launch gate to operating control

A launch test asks, “Did it pass?” while a working control asks, “What does today’s evidence cover, and is our use still within it?” The first answer ages as the model, data, and work change. The seven parts of the loop give the owner a way to keep checking that second question and change allowed use when the answer shifts.

The habit the loop builds is the one that transfers: treating “the system works” as a claim with a date on it. The loop is the machinery that keeps the date current.

The model and the work will keep changing. The loop cannot tell the team everything about the system, but it can show which cases it last checked, what changed afterward, and who must decide whether use should continue. A passing result then becomes a dated decision the owner can revisit, not a promise that will hold on its own.

Notes and sources

- [1] Chloe Autio, Reva Schwartz, Jesse Dunietz, Shomik Jain, Martin Stanley, Elham Tabassi, Patrick Hall, and Kamie Roberts, "Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile," NIST AI 600-1, July 26, 2024. <https://doi.org/10.6028/NIST.AI.600-1>
- [2] Blake Bullwinkel, Amanda Minnich, Shiven Chawla, Gary Lopez, Martin Pouliot, Whitney Maxwell, Joris de Gruyter, Katherine Pratt, Saphir Qi, Nina Chikanov, Roman Lutz, Raja Sekhar Rao Dheekonda, Bolor-Erdene Jagdagdorj, Eugenia Kim, Justin Song, Keegan Hines, Daniel Jones, Giorgio Severi, Richard Lundeen, Sam Vaughan, Victoria Westerhoff, Pete Bryan, Ram Shankar Siva Kumar, Yonatan Zunger, Chang Kawaguchi, and Mark Russinovich, "Lessons From Red Teaming 100 Generative AI Products," Microsoft Research, January 2025. <https://www.microsoft.com/en-us/research/publication/lessons-from-red-teaming-100-generative-ai-products/>
- [3] D. Sculley, Gary Holt, Daniel Golovin, et al., "Hidden Technical Debt in Machine Learning Systems," Advances in Neural Information Processing Systems 28 (NeurIPS 2015). <https://proceedings.neurips.cc/paper/2015/file/86df7dcfd896fcdf2674f757a2463eba-Paper.pdf>
- [4] Ziwei Ji, Nayeon Lee, Rita Frieske, et al., "Survey of Hallucination in Natural Language Generation," ACM Computing Surveys, 2023. <https://arxiv.org/abs/2202.03629>
- [5] Raja Parasuraman and Dietrich H. Manzey, "Complacency and Bias in Human Use of Automation: An Attentional Integration," Human Factors 52, no. 3 (2010): 381–410. <https://depositonce.tu-berlin.de/bitstreams/cafd2873-814b-4c59-bab1-addd42e249d2/download>

About the author

Marco Policani is an enterprise portfolio, PMO, and AI operating-governance leader. He builds portfolio governance systems where AI carries the analytical load and named humans own every decision — an approach documented across case studies, walkthroughs, and working governance guides on his portfolio site.

Portfolio & governance library: policani.net/governance · Full portfolio: policani.net · LinkedIn: linkedin.com/in/marcpolicani

This white paper may be shared freely with attribution. © 2026 Marco Policani.